Pluralsight

AI ENGINEERING ACADEMY · WEEK 2

# Serving in Agile Teams + Collaboration

Week 2 of 24 - Sprint practices, team roles, Git & the eComMART AI Transformation Story

**Mudit Khetan**
Pluralsight Instructor

# Today's Agenda

Everything we cover today, from agile team roles to your first pull request.

## Agile Team Roles

Cross-functional teams, Scrum roles, shared ownership.

## Sprint Practices

Planning, Daily Scrum, Refinement, Review, Retrospective.

## Story Evolution

Watch a raw idea become a sprint-ready story in 5 steps on the Canva board.

## Git & Collaboration

Account, repo, branch, commit, pull request, merge - live demo.

## Feedback & Communication

Bad vs. good feedback at every sprint stage.

## AI-Ready Habits

Four habits every team needs when AI is part of the product.

# Day 1 of Sprint 1 - What Went Wrong?

The code was not bad. The process was. By 5 pm the agent counters at $65 on an $80 floor.

## No shared plan

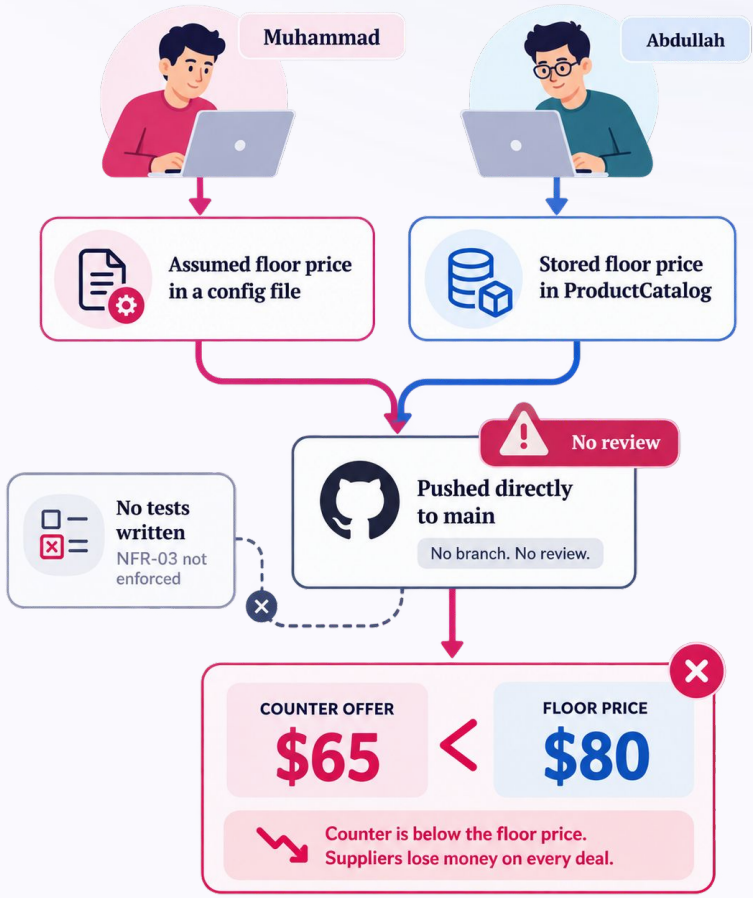
Muhammad and Abdullah assumed different things about the floor-price data. Nobody checked.

## No tests written

NFR-03 said no counter below the floor price. Nobody wrote a test to enforce it.

## Both pushed directly to main

No branch. No review. One bad assumption went live and cost suppliers money on every deal.



# Quick Poll
# What have we learnt so far



# How an Agile Team Works

- Cross-functional roles and why silos fail
- The three Scrum roles and what each one owns
- Shared ownership vs. silo thinking
- Accountability without blame



# Cross-Functional Team: All Skills in One Team

The team holds every skill needed to finish a story. No queues. No handoffs.

### Product Owner (PO)

Owns the backlog and decides what matters most.

### Scrum Master (SM)

Guards the process and clears the squad's blockers.

### DEV

Builds, tests, and demos a working slice each sprint.

#### Developer + AI Engineer

Writes the code. Builds the AI prompt. Owns counter_offer().

#### DATA

Prepares and manages data. Owns the ProductCatalog.

#### EVAL + QA

Evaluates AI behaviour and writes tests. Owns NFR checks.

#### UX + SEC

Designs the experience. Keeps secrets and API keys safe.

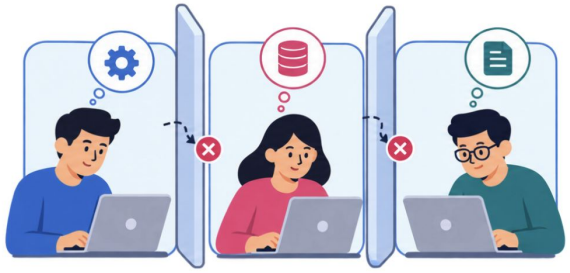
***Cross-functional does not mean no dependencies.*** *It means dependencies are visible, handled in the same conversation.*

# Silo Thinking vs. Shared Ownership

Shared ownership: the team owns the Sprint Goal together. Done = the whole story works.

## Silo thinking

- "My function works. I am done."
- "QA will catch it."
- "That is not my task."
- Hide a problem until the demo.



## Shared ownership

- "The story is delivered, not just worked."
- "We build quality in. QA helps us prove it."
- "What does the Sprint Goal need next?"
- Raise it at the next Daily Scrum.



***Accountability is not blame.*** *Ask "what in our process let this through?" not "who did this?"*

# Silo Thinking or Shared Ownership?

Type S (Silo) or O (Ownership) in the chat for each scenario.

| | |
|---|---|
| Abdullah finishes building ProductCatalog and closes his task without checking whether counter_offer() has the data it needs. | Silo Thinking |
| Sam discovers that the floor-price test is failing. He raises the blocker during the Daily Scrum, and the team adjusts its plan. | Shared Ownership |
| A developer says, “My function works. Testing is QA’s responsibility.” | Silo Thinking |
| An EVAL engineer finishes testing the AI and logs edge cases so QA can verify them. | Shared Ownership |
| A build breaks, and the team asks, “Did we have a test for this case, and how can we prevent it from happening again?” | Shared Ownership |

# Story Evolution: From Raw Idea to Sprint-Ready

- Stage 1: Basic story (As a seller agent, I want to...)
- Stage 2: Add acceptance criteria (Given / When / Then)
- Stage 3: Split the story into two smaller stories
- Stage 4: Size it (Fibonacci) + Stage 5: Add Definition of Done



# Story Sizing: What, Why, and How

Story points capture effort, complexity, and uncertainty together - not hours.

| Story Point | EFFORT | RESOURCE | RISK | COMPLEXITY |
|---|---|---|---|---|
| 1 | Minimum | A few minutes | None | Little |
| 2 | Minimum | A few hours | None | Little |
| 3 | Mild | A day | Low | Low |
| 5 | Moderate | A few days | Moderate | Medium |
| 8 | Severe | A week | Moderate | Medium |
| 13 | Maximum | A month | High | High |

### WHAT: Story Points

A relative measure of size based on effort, resources, risk and complexity (not actual hours).

### WHY: It Matters

Helps the team have a shared understanding and better conversations during planning.

### HOW: Fibonacci Scale

Use 1 · 2 · 3 · 5 · 8 · 13. Gaps grow on purpose. If a story feels like 20+, split it.

### Planning Poker

Everyone reveals their estimate at the same time. Stops one loud voice from anchoring the group.

**Any story of size 13 should be broken down into shorter stories or turned into an (sub)epic.**



# Story Evolution: From Raw Idea to Testable Story

Turning a business need into clear, verifiable requirements.

STARTING POINT

## Business Need

**“Suppliers spend too much time negotiating low offers.”**

DISCUSS

**What value are we trying to deliver?**

STAGE 1 · STORY 2.2

## User Story

*“As a supplier, I want low offers countered automatically, so I save negotiation time.”*

| | |
|---|---|
| WHO | **Supplier** |
| WHAT | **Automatic counter-offers for low offers** |
| WHY | **Save negotiation time** |

STAGE 2

## Acceptance Criteria

| | |
|---|---|
| GIVEN | A product has an N80 floor price. |
| WHEN | A buyer offers N70. |
| THEN | The counter must never fall below N80, and the decision must be logged. |

**Acceptance criteria turn expectations into behaviour the team can test.**

# Making Story 2.2 Sprint-Ready

Refine the scope, estimate the effort, and agree what Done means.

STAGE 3

## Split the Story

Is the story too large, or does it combine work that could be delivered independently?

**Possible slice A**
Core counter-offer and floor-price behaviour

**Possible slice B**
Stock-aware handling, exceptions and logging

*Illustrative example, not an approved split.*

**Can each proposed story be delivered and tested independently?**

*The team decides whether splitting is justified.*

STAGE 4

## Estimate the Work

| 1 | 2 | 3 | 5 | **8** | 13 |
|---|---|---|---|---|---|

**Story 2.2 = 8 story points**

Story points estimate relative effort, complexity, and uncertainty - not hours.

**Would you keep the story at 8 points or split it? Discuss before voting.**

*If split, the team estimates each new story.*

STAGE 5

## Definition of Done

- [ ] Code reviewed and merged.
- [ ] Tests written and passing.
- [ ] Demonstrated to the Product Owner.

**Working code alone does not mean the story is Done.**

**Refine the story** → **Prepare it for Sprint Planning** → **Track tasks on the Sprint board**

*Story refinement prepares the work; the Sprint board tracks the tasks.*

# Sprint Practices

- Planning · Daily Scrum · Refinement
- Sprint Review · Retrospective
- Dependencies, blockers and the Kanban board
- Story sizing and planning poker



# The Sprint at a Glance: Five Events, Every Sprint

One rhythm repeated until the product is done. Plan → Build → Review → Improve.



### Backlog Refinement (ongoing)

Make upcoming stories clear, sized, and ready. Output: stories that meet the Definition of Ready.



### Sprint Planning (4 hrs)

PO proposes the goal. Team selects stories, breaks them into tasks, spots dependencies.



### Daily Scrum (15 min)

Sync and adapt the plan for the next 24 hours. Blockers surfaced.



### Sprint Review (2 hrs)

Demo Done stories to PO and stakeholders. Inspect the product. Adjust the backlog.



### Retrospective (1.5 hrs)

Blameless conversation about process. Stop / Continue / Start. One concrete action.

***Rhythm:*** *Refine → Plan → Build (Daily Scrum) → Review → Retro, then repeat with what you learned.*

# Managing Dependencies and Blockers

A dependency is something you need before you can finish. A blocker is stopping work right now.

## Dependency type

- Task to task
- Data
- Decision
- Access or tools

## eComMART example

- **counter_offer()** needs stock count from ProductCatalog
- Supplier floor prices must exist before the floor-price test can run
- PO must define what counts as an unusual request
- Team needs a shared repo before anyone can open a pull request




***Dependency vs. blocker:*** *a dependency is planned around; a blocker stops work right now - raise it the same day.*

# Retrospective: Stop / Continue / Start

Blameless conversation about process. Must end with specific actions the team will actually try.

### STOP

Merging without a review. Pushing directly to main. Hiding a problem until the demo.

### CONTINUE

Holding the Daily Scrum at the same time every day. Writing AC before coding.

### START

Writing the floor-price test before the function. Reviewing a card when In Review is full.

### Remember

The retro inspects HOW we built it. The review inspects WHAT we built. They are not the same event.

***Mindset first:*** *Agile is a way of thinking about uncertainty, not just a set of meetings.*

# Sprint Review or Retrospective?

Type R (Review) or T (Retrospective) in the chat for each scenario.

| | |
|---|---|
| The team demonstrates the Seller Agent to stakeholders and collects feedback on its features. | **Sprint Review** |
| The team discusses why two stories were not completed and agrees to plan a buffer for the next sprint. | **Sprint Retrospective** |
| A stakeholder suggests changing the Seller Agent’s response to unusual customer requests. The Product Owner updates the backlog. | **Sprint Review** |
| The team identifies that testing starts too late and agrees to write tests before implementing functions. | **Sprint Retrospective** |
| The team reflects on communication problems during the sprint and agrees on one improvement for next time. | **Sprint Retrospective** |

# Demo

## Git and Collaboration

- Why Git? The chaos without version control
- Key concepts: repo, branch, commit, PR, merge
- The Git flow: branch to pull request to merge
- Live demo: GitHub account to first pull request



# Why Git? Two People, One File, Broken Agent

Muhammad saves counter_offer.py at 3pm. Abdullah saves his version at 4pm. Abdullah's overwrites Muhammad's. The floor-price logic disappears.

### Full history

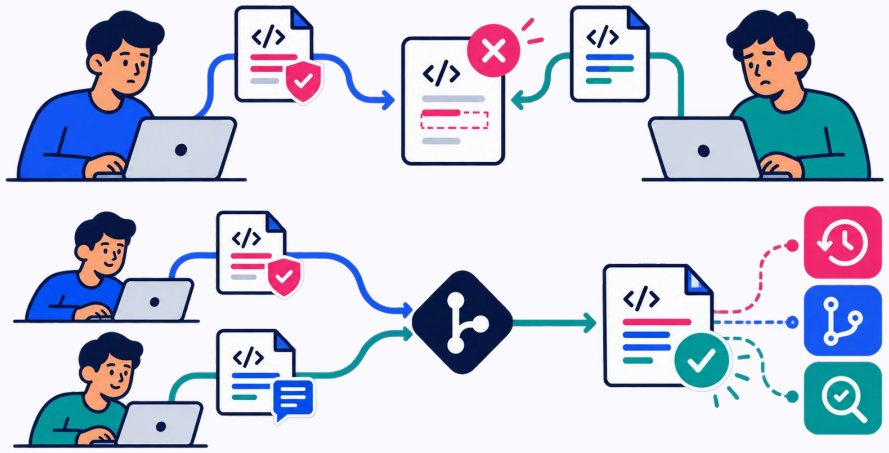
Every change is saved with who made it and why. You can always return to any checkpoint.

### Branches

Work on a feature safely. A broken branch does not touch what everyone else is using.

### Pull Requests

A teammate reads your change before it goes live. This is how the floor-price bug gets caught before it reaches buyers.



# Git: Key Concepts

| Term | Plain meaning |
| --- | --- |
| **Repository (repo)** | The project folder Git tracks — all files and their full history. |
| **Branch** | A separate copy of the project. Work safely without touching `main` |
| **Commit** | A saved snapshot with a short message. A checkpoint you can return to. |
| **Push** | Send your commits from your computer up to GitHub. |
| **Pull** | Download the latest changes from GitHub. |
| **Pull Request (PR)** | Ask a teammate to review your branch before it merges into `main` |
| **Merge** | Join the approved branch into `main` so everyone gets the changes. |

## Typical collaboration workflow



# Feedback and Communication

- Specific · Kind · Actionable
- Bad vs. good at every sprint stage
- Planning · Daily Scrum · Refinement · Review · Retro
- Communication habits that keep a cross-functional team aligned



# Bad vs. Good Feedback - Across All Sprint Events

Good feedback is about the work, not the person. Specific, kind, and actionable.

| Sprint Event | Unhelpful | Helpful |
|---|---|---|
| **Sprint Planning** | ✕ "This story is too vague." | ✓ "AC-2 does not say what counts as a valid counter. Can we add a rule before we commit?" |
| **Daily Scrum** | ✕ "You should have done that yesterday." | ✓ "That task has been In Progress for 3 days. Can I help unblock you after the standup?" |
| **Backlog Refinement** | ✕ "Split it somehow." | ✓ "If we split at the stock-check boundary, Story A stays at 5 pts and Story B drops to 3." |
| **Sprint Review & Retro** | ✕ "Someone kept breaking the build." (Retro) | ✓ "Two builds broke from unreviewed merges. Stop: require a passing test before merging." (Retro) |

***Good feedback is about the work, not the person.*** *Specific, kind, and actionable.*

# AI-Ready Habits

- Four habits every team needs when AI is part of the product
- Experiment · Log failures · Measurable criteria · Human in the loop
- These habits are not extra work
- They are what stops a 3-line bug from costing suppliers real money



# The 4 AI-Ready Habits

An agent that makes mistakes needs a team that can catch them fast.

### Habit 1: Run Experiments

"If the agent uses inventory data, hallucinated stock replies drop below 2%."

### Habit 2: Log Failure Cases

When the AI gets it wrong, write it down immediately. Every invented stock number is an NFR-02 violation - log it.

### Habit 3: Measurable Success Criteria

Every AI story needs a number. "Floor price correct 95% of the time. No fabricated prices 100%. Reply under 5 seconds."

### Habit 4: Human in the Loop

For high-stakes decisions, a human reviews before the action is final. Any counter-offer over N500,000 flags for a human agent to confirm.

***Remember:*** *a requirement for AI should always include how we will evaluate it.*

# Quick Poll
# What have we learnt so far





Pluralsight

# Thank You

See You Next Week: Python Foundations + eComMART Data Layer

# DEMO 1 of 4: Create Account and Repository

Instructor screen share starts here.

### Step 1

Go to github.com, sign up and verify email.

### Step 2

Click + New repository, name it ecommart-seller-agent.

### Step 3

Tick Initialise with README and click Create repository.




***Click the green Code button and copy the HTTPS URL.***

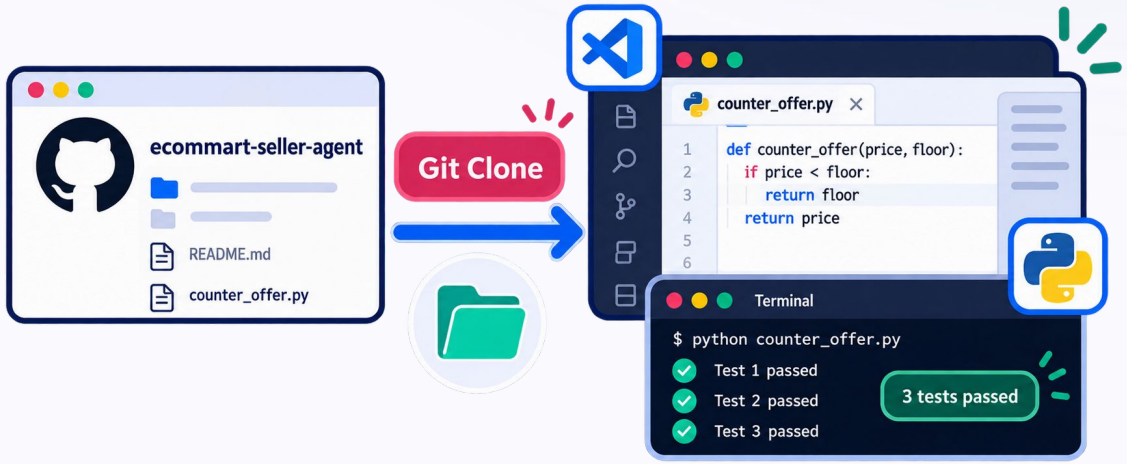
# DEMO 2 of 4: Clone to VS Code and Create First File

VS Code is open. Git is installed.

### Step 1
Ctrl+Shift+P in VS Code, type Git: Clone, paste the URL.

### Step 2
Open the cloned folder. Open the terminal with Ctrl + backtick.

### Step 3
Create counter_offer.py and type the function from the guide.




***Run: python counter_offer.py and see the three test results.***

# DEMO 3 of 4: Stage, Commit, Push

The file is saved. Now record it in Git history.

### Step 1

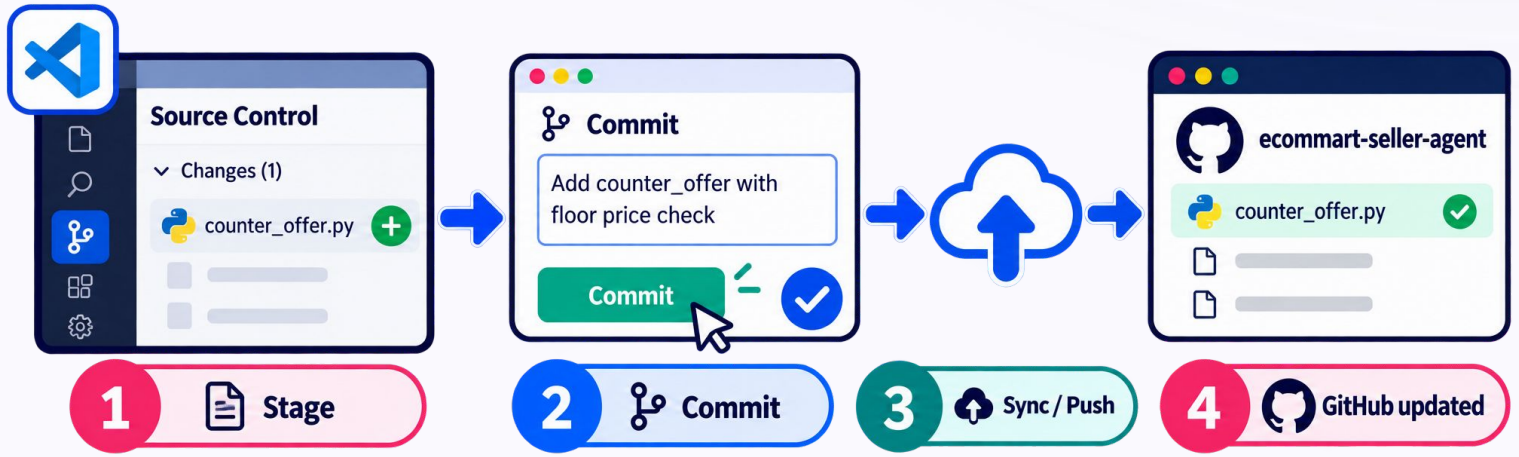
Click the Source Control icon (branching icon) in VS Code sidebar.

### Step 2

Hover counter_offer.py and click + to stage it.

### Step 3

Type commit message: Add counter_offer with floor price check.




***Click Commit then Sync Changes. Refresh GitHub to see the file.***

# DEMO 4 of 4: Branch, PR, Review, Merge

Adding the stock=0 fix for Story 2.2 AC-3.

### Step 1

In terminal: git checkout -b feature/story-2-2-stock-check

### Step 2

Add stock=None parameter and the out-of-stock return block.

### Step 3

git add counter_offer.py, commit, git push -u origin feature/...




***GitHub: Compare and pull request, fill the template, Create PR, review, Merge. Delete the branch.***

# Flashcard 3: What Is Wrong with This Commit Message?

Type your improved version of the bad commit message in the chat.

| | |
|---|---|
| git commit -m "fix stuff" | Bad |
| git commit -m "Add stock=0 check - fixes AC-3 (no invented availability)" | Good |
| git commit -m "changes" | Bad |
| git commit -m "Add counter_offer with floor price check" | Good |
| git commit -m "Prevent offers below the supplier floor price" | Good |